

Anomaly Detection Results

Part 1: Baseline Results

After running the notebook, copy the baseline results here:

Metric	Value
Total data points	1050
Anomalies injected	50
Anomalies detected by the model	50
True positives (correctly flagged)	39
False positives (normal data flagged)	14
False negatives (anomalies missed)	11
Precision	0.736
Recall	0.780
F1 Score	0.757

Scatter Plot Observations

Your observations here

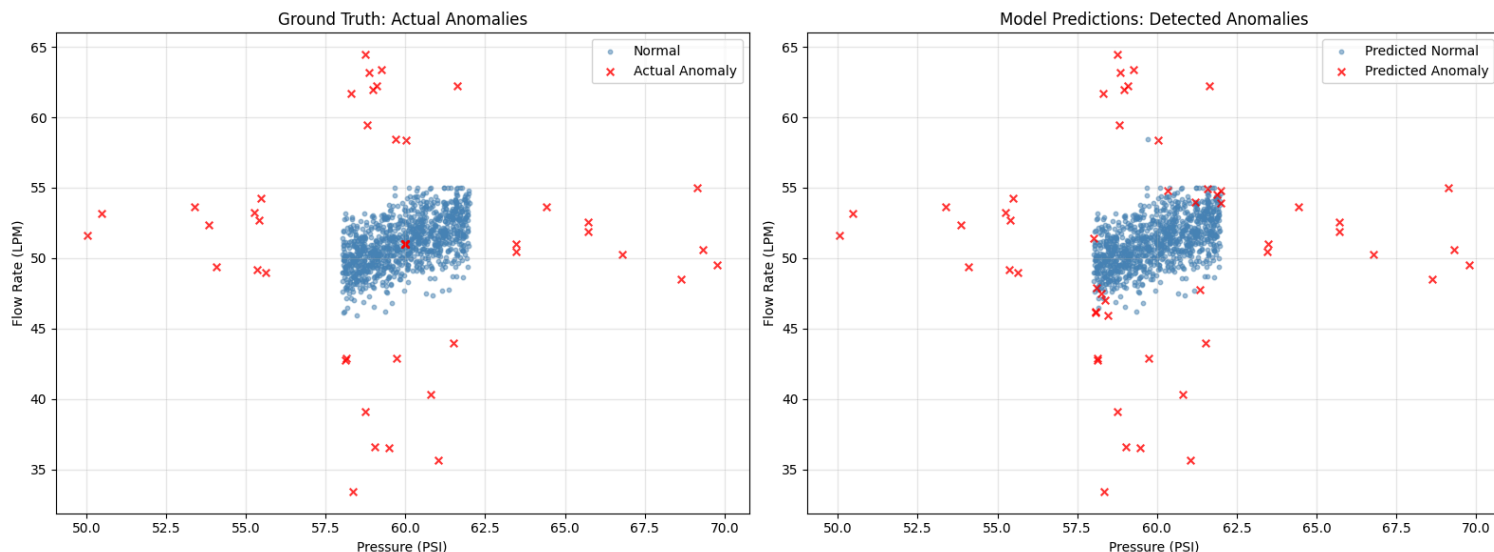
Left plot: where we actually put anomalies.

Right plot: where the model thinks anomalies are.

In the left plot, the actual anomalies are less than around the normal data.

In the right plot, the predicated anomaly is near the cluster of predicated normal data.

The left plot shows that the actual anomalies are much fewer than the normal observations and are mostly located outside the main data cluster. In the right plot, several predicted anomalies appear close to or inside the cluster of predicted normal data, suggesting that the model may produce some false positives.



Parameter Experiment

You changed **contamination** from **0.05** to **0.1**. Record the comparison below:

Metric	Default (0.05)	Changed (0.1)
Anomalies detected	53	105
True positives	39	40
False positives	14	65
Precision	0.736	0.381
Recall	0.780	0.800
F1 Score	0.757	0.516

Write one sentence explaining what changed when you increased contamination:

When contamination increased from 0.05 to 0.10, the model flagged more observations as anomalies. True positives increased only slightly, from 39 to 40, while false positives rose sharply from 14 to 65. Recall improved slightly, but precision fell from 0.736 to 0.381, and the F1 score dropped from 0.757 to 0.516. This shows that the higher contamination setting made the model more sensitive but much less precise, causing many more normal observations to be incorrectly classified as anomalies.

Part 2: Run LOF with Default parameters.

n_neighbors	Precision	Recall	F1	Time
10	0.755	0.800	0.777	0.011s
20	0.755	0.800	0.777	0.008s
50	0.755	0.800	0.777	0.015s

Best n_neighbors by F1: 10 (F1 = 0.777)

Which `n_neighbors` value gives the best F1 score?

All `n_neighbors` values give the same score 0.777, so there is no difference between them.

Part 4: Your Recommendation

- Write a 3-5 sentence recommendation in the doc answering this question: **Which model should the Grand Marina deploy for real-time anomaly detection on their water system, and why?**
- Consider:
 - Detection performance (precision, recall, F1)
 - Speed and real-time capability
 - How easy it is to integrate into the existing subscriber
 - False alarm tolerance (the night shift has to respond to every alert)

All `n_neighbors` values give the same score 0.777, so there is no difference between them.

For detection performance, we need precision , recall and F1 to help us find the anomalies pattern from the predicated model and actual model.

Grand Marina should deploy the Isolation Forest model for real-time anomaly detection. Although LOF achieved a slightly higher F1 score of 0.777, changing its `n_neighbors` value did not improve performance, and Isolation Forest is better suited to scoring new MQTT messages individually as they arrive. It is also easier to integrate into the existing subscriber and provides a practical balance of precision, recall, and speed. Because night-shift employees must investigate every alert, the hotel should use the 0.05 contamination setting to reduce false alarms while maintaining strong detection performance.